

Stream, per intero

Pipeline, laziness, ordine di valutazione,
collettori e l'anatomia della libreria



prof. Massimiliano Redolfi

Università degli Studi di Brescia · Ingegneria Informatica

massimiliano.redolfi@unibs.it

Versione 0.1

Quest'opera è distribuita con licenza [Creative Commons](#)
“Attribuzione – Non commerciale – Condividi allo stesso
modo 4.0 Internazionale”.



Indice

3	Stream, per intero	1
3.1	Un flusso di dati	1
3.2	Come lavora una pipeline	3
3.3	Creare stream	6
3.4	La metà mancante: raccogliere	9
3.5	Stream o ciclo?	12
3.6	Dentro la libreria	13
3.7	Riepilogo	19
3.8	Esercizi	20
	Soluzioni	21

Stream, per intero

Pipeline, laziness, ordine di valutazione, collettori e l'anatomia della libreria.

Il punto di partenza è un ciclo qualunque: scorrere una lista, tenere alcuni elementi, trasformarli, ordinarli, stamparli. Il punto di arrivo è un modo diverso di scrivere lo stesso lavoro, come una *pipeline* in cui i dati scorrono attraverso una sequenza di operazioni, ciascuna delle quali è una lambda. Le pipeline si leggono bene, ma per usarle con sicurezza bisogna sapere *come* vengono eseguite: quando parte il lavoro, in che ordine gli elementi attraversano le operazioni, dove la libreria si ferma in anticipo e dove invece è costretta a vedere tutto. La seconda parte della dispensa apre la libreria `java.util.stream` e mostra le classi che realizzano questo meccanismo, con qualche considerazione su ciò che costa e ciò che non costa.

Le lambda expression, i method reference e le interfacce di `java.util.function` sono l'argomento di una dispensa a parte: qui le useremo come linguaggio delle pipeline, dandole per note.

3.1 Un flusso di dati

Da una lista di stringhe vogliamo stampare, in ordine alfabetico e in maiuscolo, quelle che iniziano per c. Con i costrutti classici:

```
List<String> myList = List.of("a1", "a2", "b1", "c1", "c2");

List<String> conC = new ArrayList<>();
for (String s : myList) {
    if (s.startsWith("c")) conC.add(s);
}
List<String> maiuscole = new ArrayList<>();
for (String s : conC) {
    maiuscole.add(s.toUpperCase());
}
Collections.sort(maiuscole);
for (String s : maiuscole) {
    System.out.println(s);
}
```

Tre cicli, due liste d'appoggio, un `if`: il *come* copre il *che cosa*. Si potrebbe fondere i primi due cicli in uno, ma l'ordinamento costringe comunque a raccogliere prima e stampare dopo, e la lista d'appoggio resta. Lo stesso programma, scritto con uno stream:

```
myList.stream()
    .filter(s -> s.startsWith("c"))
    .map(String::toUpperCase)
    .sorted()
    .forEach(System.out::println);
```

C1
C2

Conviene immaginare i dati che *scorrono dall'alto*: a ogni riga vengono filtrati, trasformati, ordinati, infine stampati. Non ci sono indici, variabili d'appoggio, né un `if` dentro un `for`. Ogni riga dice una cosa sola, e la si legge nell'ordine in cui la si pensa.

3.1.1 La pipeline

Definizione 3.1: Stream e pipeline

Uno *stream* è una sequenza di elementi che si può attraversare *una volta*, sulla quale si applicano operazioni in cascata. Una *pipeline* è composta da tre parti:

- una **sorgente**, che produce gli elementi: una collezione, un array, un intervallo di interi, le righe di un file;
- zero o più **operazioni intermedie**, ciascuna delle quali riceve uno stream e ne restituisce un altro, così che si possano concatenare;
- una **operazione terminale**, che consuma lo stream e produce un risultato (un valore, una collezione) o un effetto (una stampa).

```
myList.stream()           // 1. sorgente
    .filter(s -> s.startsWith("c")) // 2. operazione intermedia
    .map(String::toUpperCase) // 2. operazione intermedia
    .sorted()             // 2. operazione intermedia
    .forEach(System.out::println); // 3. operazione terminale
```

Osservazione 3.1

Uno stream *non è una collezione*. Non contiene dati: li fa passare. Non si può chiedergli quanti elementi ha senza consumarlo, non lo si può attraversare due volte, non ha un metodo `get(i)`. La collezione è la sorgente; lo stream è il trasporto.

3.1.2 Le operazioni

Le operazioni sono molte, ma quelle che si usano davvero stanno in una tabella. La colonna di sinistra restituisce uno `Stream`, la colonna di destra un risultato.

Intermedie ($\rightarrow$ Stream)	Terminali ($\rightarrow$ risultato)
<code>filter(Predicate)</code>	<code>forEach(Consumer)</code>
<code>map(Function), mapToInt(...)</code>	<code>collect(Collector), toList()</code>
<code>sorted(), sorted(Comparator)</code>	<code>count(), sum(), average()</code>
<code>distinct(), limit(n), skip(n)</code>	<code>anyMatch, allMatch, noneMatch</code>
<code>flatMap(Function)</code>	<code>findFirst(), min, max</code>
<code>peek(Consumer)</code>	<code>reduce(...)</code>

Quasi tutte ricevono una lambda: un `Predicate` per decidere, una `Function` per trasformare, un `Consumer` per agire, un `Comparator` per ordinare. Le interfacce funzionali sono il linguaggio con cui si dice a ogni operazione che cosa fare di ciascun elemento.

3.2 Come lavora una pipeline

Le regole di questa sezione si scoprono con un solo strumento: una lambda che stampa qualcosa prima di fare il suo lavoro. Per non ripetere il codice usiamo un metodo di servizio che stampa una fase e la stringa ricevuta, e restituisce la stringa così com'è:

```
static String log(String fase, String s) {
    System.out.println(fase + ": " + s);
    return s;
}
```

3.2.1 Laziness: senza terminale non succede niente

```
Stream.of("alfa", "beta", "gamma", "delta")
    .filter(s -> {
        System.out.println("Filtro: " + s);
        return true;
    });
```

(nessun output)

Il filtro non viene mai chiamato. Le operazioni intermedie *descrivono* il lavoro; è l'operazione terminale a farlo partire. Fino ad allora lo stream è una ricetta, non un'esecuzione. Questa proprietà si chiama *laziness* (valutazione pigra) ed è la chiave di tutto ciò che segue: se il lavoro parte solo al momento della terminale, la libreria conosce *l'intera pipeline* prima di eseguire il primo passo, e può organizzarlo nel modo più conveniente.

3.2.2 Elaborazione verticale

Aggiungiamo la terminale e osserviamo l'ordine delle stampe:

```
Stream.of("alfa", "beta", "gamma", "delta")
    .filter(s -> { log("Filtro", s); return true; })
    .forEach(s -> log("For", s));
```

```
Filtro: alfa
For: alfa
Filtro: beta
For: beta
Filtro: gamma
For: gamma
Filtro: delta
For: delta
```

Non “prima tutti i filtri, poi tutti i for”: *ogni elemento attraversa tutta la pipeline* prima che parta il successivo. L’elaborazione è *verticale*, elemento per elemento, non orizzontale, operazione per operazione. Non esiste, fra `filter` e `forEach`, una lista intermedia con i quattro elementi filtrati: ogni elemento che supera il filtro viene consegnato subito all’operazione seguente.

3.2.3 L’ordine conta

Se ogni elemento attraversa le operazioni nell’ordine in cui sono scritte, l’ordine in cui le scriviamo determina quanto lavoro si fa. Confrontiamo due pipeline che producono lo stesso risultato:

```
Stream.of("alfa", "beta", "gamma")
    .map(s -> log("Map", s).toUpperCase())
    .filter(s -> log("Filter", s).startsWith("A"))
    .forEach(s -> log("For", s));
```

```
Map: alfa      Filter: ALFA      For: ALFA
Map: beta      Filter: BETA
Map: gamma     Filter: GAMMA
```

Sei chiamate di lambda: tre `map`, tre `filter`. Con le due operazioni scambiate:

```
Stream.of("alfa", "beta", "gamma")
    .filter(s -> log("Filter", s).startsWith("a"))
    .map(s -> log("Map", s).toUpperCase())
    .forEach(s -> log("For", s));
```

```
Filter: alfa   Map: alfa      For: ALFA
Filter: beta
Filter: gamma
```

Quattro chiamate: `map` lavora solo sull’elemento che ha superato il filtro. La regola pratica è *filtrare prima, trasformare dopo*: ogni elemento scartato presto è un elemento su cui le operazioni successive non lavorano.

3.2.4 Short-circuit: si elabora solo ciò che serve

Alcune operazioni possono *concludere* il lavoro prima di aver visto tutti gli elementi:

```
Stream.of("alfa", "beta", "gamma", "delta")
    .map(s -> log("Map", s).toUpperCase())
    .anyMatch(s -> log("Match", s).startsWith("B"));
```

```
Map: alfa
Match: ALFA
Map: beta
Match: BETA
```

`anyMatch` si ferma al primo `true`: `gamma` e `delta` non vengono mai toccati, né da `map` né da `anyMatch`. Le operazioni con questa proprietà si chiamano *short-circuit*: fra le terminali `anyMatch`, `allMatch`, `noneMatch`, `findFirst`, `findAny`; fra le intermedie `limit` e `takeWhile`, che dopo un certo punto non lasciano più passare nulla e lo dicono alla sorgente. Unite alla *laziness* e all'elaborazione verticale, permettono di lavorare su sorgenti enormi, o addirittura infinite, toccando solo gli elementi necessari.

3.2.5 sorted lavora in orizzontale

C'è un'operazione che non può essere verticale: per emettere il primo elemento in ordine, `sorted` deve averli visti *tutti*.

```
Stream.of("gamma", "alfa", "beta", "epsilon", "delta")
    .sorted((s1, s2) -> { log("Sort", s1 + ";" + s2); return s1.compareTo(s2);
    })
    .filter(s -> log("Filter", s).startsWith("a"))
    .map(String::toUpperCase)
    .forEach(s -> log("ForEach", s));
```

```
Sort: alfa;gamma
Sort: beta;alfa
...                (8 confronti in tutto, poi:)
Filter: alfa
ForEach: ALFA
Filter: beta
Filter: delta
Filter: epsilon
Filter: gamma
```

Prima escono tutti i confronti, poi ricomincia l'elaborazione verticale sugli elementi ordinati. `sorted` è una *barriera*: raccoglie tutto, ordina, e solo allora rilascia gli elementi uno alla volta alle operazioni successive.

Definizione 3.2: Operazioni stateless e stateful

Un'operazione intermedia è *stateless* se il trattamento di un elemento non dipende dagli elementi visti prima: `filter`, `map`, `flatMap`, `peek`. È *stateful* se deve ricordare qualcosa: `distinct` (gli elementi già emessi), `skip` e `limit` (quanti ne sono passati), `sorted` (tutti gli elementi, prima di emetterne uno).

Il costo di una barriera non è solo il tempo dell'ordinamento: è la memoria per tenere tutti gli elementi, e la perdita dei vantaggi della laziness a monte. Con il `filter prima` di `sorted`, nell'esempio, arriverebbe all'ordinamento un solo elemento, e non ci sarebbe alcun confronto da fare.

3.2.6 Uno stream si consuma una volta sola

```
IntStream s = IntStream.iterate(2, f -> 2 * f);
s.anyMatch(f -> f > 15);
s.anyMatch(f -> f > 32);
```

```
Exception in thread "main" java.lang.IllegalStateException:
    stream has already been operated upon or closed
```

La seconda chiamata fallisce: la prima operazione terminale ha *consumato* lo stream. Non è una limitazione arbitraria. Uno stream è un trasporto, non un deposito: può essere collegato a una sorgente che non si può riavvolgere (le righe di un file, un generatore infinito) e non conserva gli elementi passati. Quando serve rifare il lavoro, serve un nuovo stream, e un `Supplier` è il modo più pulito per fabbricarlo a richiesta:

```
Supplier<IntStream> supplier = () -> IntStream.iterate(2, f -> 2 * f);
supplier.get().anyMatch(f -> f > 15);
supplier.get().anyMatch(f -> f > 32);
```

Per una collezione basta chiamare di nuovo `stream()`: ogni chiamata restituisce uno stream nuovo sulla stessa sorgente.

3.3 Creare stream

La sorgente più comune è una collezione, ma non è l'unica:

```
myList.stream()                // da una collezione
Stream.of("alfa", "beta", "gamma") // da valori
Arrays.stream(new int[] {1, 2, 3}) // da un array
IntStream.range(1, 10)         // 1..9
IntStream.rangeClosed(1, 10)   // 1..10
Stream.iterate(1, n -> n * 2)   // infinito: 1, 2, 4, 8, ...
Files.lines(Path.of("dati.txt")) // le righe di un file
```

`Files.lines` è l'unico caso in cui lo stream tiene aperta una risorsa del sistema: va chiuso, e il modo giusto è il `try-with-resources`, perché `Stream` implementa `AutoCloseable`. Gli altri stream non hanno nulla da chiudere.

3.3.1 Stream infiniti

`Stream.iterate( seme, f )` produce il seme, poi `f( seme )`, poi `f( f( seme ) )`, senza mai fermarsi. È utile proprio perché non decide da solo quando finire: lo decide la pipeline.

Attenzione 3.1

Uno stream infinito va sempre chiuso da `limit`, da `takeWhile` o da un'operazione terminale short-circuit come `findFirst` o `anyMatch`. Un `forEach`, un `sorted` o un `count` su uno stream infinito non terminano mai: `sorted` e `count` devono vedere tutti gli elementi, e gli elementi non finiscono.

Esempio 3.1: Fibonacci con `iterate`

La successione di Fibonacci si scrive di solito con un ciclo e due variabili, o con una ricorsione. Con `iterate` lo *stato*, cioè la coppia di termini correnti, viaggia dentro l'elemento: ogni elemento è un array di due `long`, e la funzione produce la coppia successiva dalla precedente.

```
LongStream fibs = Stream
    .iterate(new long[] {1, 1}, f -> new long[] {f[1], f[0] + f[1]})
    .mapToLong(f -> f[0]);

fibs.limit(10).forEach(System.out::println);
```

```
1 1 2 3 5 8 13 21 34 55
```

Lo stream è infinito; `limit(10)` lo chiude. Se al posto di `limit` ci fosse `takeWhile(f -> f < 100)` **JAVA 9**, la pipeline si fermerebbe al primo termine che supera cento.

Dalla versione 9 `iterate` ha anche una forma a tre argomenti, con un predicato di continuazione, che rende lo stream finito alla maniera di un ciclo `for`: `IntStream.iterate(1, n -> n < 100, n -> n * 2)` produce le potenze di due minori di cento.

3.3.2 Stream primitivi

`Stream<T>` è generico: i suoi elementi sono oggetti. Per i numeri esistono tre stream dedicati, `IntStream`, `LongStream` e `DoubleStream`, che lavorano direttamente sui tipi primitivi. Hanno operatori numerici che `Stream<Integer>` non ha (`sum`, `average`, `range`) ed evitano il *boxing*, la conversione avanti e indietro tra `int` e `Integer` che altrimenti accompagnerebbe ogni elemento a ogni passo.

```

IntStream.range(1, 10).filter(v -> v % 2 == 0).sum();           // 20

Arrays.stream(new int[] {1, 2, 3}).map(n -> n * n).average(); //
OptionalDouble[4.67]

Stream.of("a1", "a2", "a3")
    .map(s -> s.substring(1))
    .mapToInt(Integer::parseInt)    // Stream<String> -> IntStream
    .max();                        // OptionalInt[3]

IntStream.range(1, 4)
    .mapToObj(i -> "a" + i)        // IntStream -> Stream<String>
    .toList();                    // [a1, a2, a3]

```

`mapToInt`, `mapToLong`, `mapToDouble` e `mapToObj` sono i *ponti* fra i due mondi; `boxed()` trasforma un `IntStream` in uno `Stream<Integer>` quando serve un collettore che lavora su oggetti. Le lambda che gli stream primitivi ricevono sono le varianti specializzate delle interfacce funzionali: `IntPredicate`, `IntUnaryOperator`, `IntFunction`, e così via.

3.3.3 Optional: un risultato che può mancare

```

Stream.of("alfa", "beta", "gamma").findFirst();           // Optional[alfa]
Stream.<String>empty().findFirst();                         // Optional.empty

```

`findFirst`, `min`, `max`, `average` e `reduce` senza valore iniziale hanno un problema in comune: su uno stream vuoto *non esiste* un risultato. Prima di Java 8 la convenzione era restituire `null` e sperare che il chiamante controllasse. `Optional<T>` rende il dubbio esplicito nel tipo.

Definizione 3.3: Optional

`Optional<T>` è un contenitore che ha *al più* un valore di tipo `T`. Un metodo che restituisce `Optional<T>` dichiara, nella firma, che il risultato potrebbe mancare, e obbliga il chiamante a dire che cosa fare in quel caso. Per i primitivi esistono `OptionalInt`, `OptionalLong` e `OptionalDouble`.

```

Optional<String> opt = myList.stream().filter(s -> s.startsWith("z")).findFirst();

opt.ifPresent(System.out::println);           // fai qualcosa se c'è
String s = opt.orElse("nessuno");             // valore di riserva
String t = opt.orElseThrow();                 // eccezione se manca
String u = opt.map(String::toUpperCase).orElse(""); // trasforma, se c'è

```

Attenzione 3.2

`opt.get()` senza un controllo preliminare è un `null` travestito: lancia `NoSuchElementException` se il valore manca, esattamente come il `null` avrebbe lanciato `NullPointerException`. Se vi trovate a scrivere `get()`, quasi sempre volevate `orElse` o `orElseThrow`, che dicono che cosa succede nel caso mancante. `Optional` è pensato per i valori di ritorno: non per i campi, non per i parametri, non per le collezioni.

3.4 La metà mancante: raccogliere

`forEach` produce *effetti*: stampe, scritture. Molto più spesso da una pipeline vogliamo un *risultato*: una lista, un insieme, una mappa, un numero. È il compito di `collect`, della classe `Collectors` che fornisce i collettori pronti, e di `reduce`.

3.4.1 `collect` e `toList`

```
List<String> maiuscole = myList.stream()
    .map(String::toUpperCase)
    .toList();                                // lista immutabile

Set<Character> iniziali = myList.stream()
    .map(s -> s.charAt(0))
    .collect(Collectors.toSet());

String tutte = myList.stream()
    .collect(Collectors.joining(", "));        // "a1, a2, b1, c1, c2"
```

`toList()` **JAVA 16** restituisce una lista *immutabile*; se serve una lista da modificare, `collect(Collectors.toList())` restituisce un `ArrayList` (la documentazione non lo garantisce, ma è così, e per una lista modificabile con garanzia c'è `collect(Collectors.toCollection(ArrayList::new))`). `toSet`, `joining`, `counting`, `summingInt`, `averagingDouble` sono altri collettori della stessa famiglia; conviene importarli con `import static java.util.stream.Collectors.*`; come faremo da qui in avanti.

Attenzione 3.3

Riempire una lista dentro un `forEach` funziona, ma è il modo sbagliato:

```
List<String> risultato = new ArrayList<>();
myList.stream().map(String::toUpperCase).forEach(risultato::add); // no
```

La lambda modifica una variabile esterna, cioè ha un effetto collaterale; con una pipeline parallela il risultato sarebbe corrotto, e comunque il lettore deve seguire due cose, la pipeline e la lista, per capire che cosa succede. Il risultato è del `collect`, non dell'effetto collaterale.

3.4.2 Che cos'è un `Collector`

Definizione 3.4: `Collector`

Un `Collector<T, A, R>` è una ricetta per accumulare elementi di tipo `T` in un contenitore di tipo `A` e ricavarne un risultato di tipo `R`. È fatto di quattro funzioni:

- `supplier`: crea un contenitore vuoto (`ArrayList::new`);
- `accumulator`: aggiunge un elemento al contenitore (`List::add`);
- `combiner`: fonde due contenitori, quando la pipeline è parallela;
- `finisher`: trasforma il contenitore nel risultato finale (spesso l'identità).

Non serve scrivere collettori a mano: `Collectors` ne fornisce per ogni caso comune, e i collettori si compongono fra loro, come vedremo con `groupingBy`. Serve però sapere come

sono fatti, perché la firma `Collector<T, A, R>` compare nella documentazione e perché, nella seconda parte della dispensa, vedremo che `collect` non è altro che una riduzione guidata da queste quattro funzioni.

3.4.3 groupingBy: da lista a mappa

```
Map<Character, List<String>> perIniziale = myList.stream()
    .collect(groupingBy(s -> s.charAt(0)));
// {a=[a1, a2], b=[b1], c=[c1, c2]}
```

`groupingBy` riceve una funzione di classificazione e mette ogni elemento nella lista associata alla sua chiave. Il secondo argomento, facoltativo, è un altro collettore e dice *che cosa fare di ogni gruppo* invece di tenerne la lista:

```
Map<Character, Long> quante = myList.stream()
    .collect(groupingBy(s -> s.charAt(0), counting()));
// {a=2, b=1, c=2}

Map<Character, Set<String>> insiemi = myList.stream()
    .collect(groupingBy(s -> s.charAt(0), toSet()));

Map<Boolean, List<String>> pari = myList.stream()
    .collect(partitioningBy(s -> s.endsWith("2")));
// {false=[a1, b1, c1], true=[a2, c2]}
```

`partitioningBy` è un `groupingBy` con un predicato al posto della funzione: le chiavi sono sempre e solo `true` e `false`, entrambe presenti anche se un gruppo è vuoto.

Esempio 3.2: Il censimento delle forme

Data una lista di forme geometriche `shapes` di tipo `List<Shape2d>`, vogliamo sapere quante ce ne sono di ciascuna classe concreta. Con una mappa e un ciclo:

```
Map<Class<?>, Integer> census = new HashMap<>();
for (Shape2d s : shapes) {
    Integer c = census.get(s.getClass());
    census.put(s.getClass(), c == null ? 1 : c + 1);
}
```

Con uno stream:

```
Map<Class<?>, Long> census = shapes.stream()
    .collect(groupingBy(Shape2d::getClass, counting()));
```

Stessa mappa, stesso contenuto. La differenza è che la seconda versione dice *che cosa* vogliamo, un raggruppamento per classe con il conteggio di ogni gruppo, e non *come* ottenerlo con un leggi-modifica-scrivi.

3.4.4 reduce: da molti a uno

```
int somma = IntStream.rangeClosed(1, 5).reduce(0, (a, b) -> a + b);    // 15
int somma = IntStream.rangeClosed(1, 5).reduce(0, Integer::sum);      // 15
int somma = IntStream.rangeClosed(1, 5).sum();                        // 15
```

`reduce(identità, operatore)` parte dal valore iniziale e combina un elemento alla volta: $((0 + 1) + 2) + \dots$. L'identità deve essere davvero neutra per l'operatore (zero per la somma, uno per il prodotto, la stringa vuota per la concatenazione), altrimenti in una pipeline parallela, dove la riduzione parte più volte, verrebbe contata più volte. Senza identità, `reduce` restituisce un `Optional`: su uno stream vuoto non c'è nulla da combinare.

```
Optional<String> piuLunga = myList.stream()
    .reduce((a, b) -> a.length() >= b.length() ? a : b);
```

`sum`, `count`, `min`, `max`, `joining` sono riduzioni già scritte: quando esistono, si usano quelle, perché dicono subito che cosa calcolano e perché la libreria può eseguirle in modo più efficiente di un `reduce` generico (§3.6.6).

3.4.5 flatMap: appiattare

`map` produce *un* elemento per ogni elemento in ingresso. Quando ogni elemento ne produce *molti*, serve `flatMap`:

```
List<String> righe = List.of("alfa beta", "gamma", "delta epsilon zeta");

List<String> parole = righe.stream()
    .flatMap(r -> Arrays.stream(r.split(" ")))
    .toList();
// [alfa, beta, gamma, delta, epsilon, zeta]
```

La lambda di `flatMap` restituisce uno stream per ogni elemento, e `flatMap` *srotola* ciascuno di questi stream nel flusso principale. Con `map` avremmo ottenuto uno `Stream<String[]>`: tre array, non sei parole. Lista di liste, righe di parole, ordini con più articoli: ogni volta che il risultato di `map` sarebbe una collezione, è un `flatMap`.

3.4.6 toMap e le chiavi duplicate

```
Map<String, Integer> lunghezze = myList.stream()
    .collect(toMap(s -> s, String::length));    // {a1=2, a2=2, b1=2, ...}
```

`toMap` riceve due funzioni, una per la chiave e una per il valore. A differenza di `groupingBy`, che per una chiave ripetuta accumula, `toMap` pretende chiavi distinte:

```
Map<Character, String> prima = myList.stream()
    .collect(toMap(s -> s.charAt(0), s -> s));
// IllegalStateException: Duplicate key a (attempted merging values a1 and a2)
```

Con due elementi per chiave bisogna dire *come fonderli*, con un terzo argomento:

```
Map<Character, String> unite = myList.stream()
    .collect(toMap(s -> s.charAt(0), s -> s, (x, y) -> x + "," + y));
// {a=a1,a2, b=b1, c=c1,c2}
```

3.5 Stream o ciclo?

Gli stream non sostituiscono i cicli: sono la scrittura giusta per una famiglia di problemi, e quella sbagliata per un'altra.

Stream	Ciclo for
trasformare, filtrare, raggruppare, contare	modificare la collezione mentre la si scorre
il risultato è un valore o una nuova collezione	servono l'indice, o due collezioni in parallelo
ogni passo è una lambda breve e senza stato	il corpo è lungo, o ha <code>break</code> e <code>continue</code> complessi

Osservazione 3.2

Se la pipeline supera le cinque o sei operazioni, o le lambda crescono oltre una riga, conviene spezzarla: un metodo con un nome per ogni pezzo, o una variabile intermedia con un nome per il risultato parziale. Uno stream illeggibile non è più moderno di un ciclo chiaro.

Gli errori più frequenti sono le violazioni delle regole viste fin qui:

- **effetti collaterali nelle lambda intermedie:** modificare una variabile esterna da `map` o `filter` non è permesso se è locale (deve essere *effectively final*) e, se è un campo o una collezione, rende la pipeline dipendente dall'ordine di esecuzione, che la libreria non promette;
- **riusare uno stream:** `IllegalStateException`; serve un `Supplier` o una nuova chiamata a `stream()`;
- **stream infinito senza limit:** il programma non termina;
- **forEach per costruire una collezione:** è un `collect`;
- **get() su un Optional** senza controllo: è un `null` con un altro nome;
- **sorted prima di filter:** si ordina anche ciò che si scarnerà.

3.5.1 E parallel()?

```
long n = shapes.parallelStream().filter(s -> s.area() > 10).count();
```

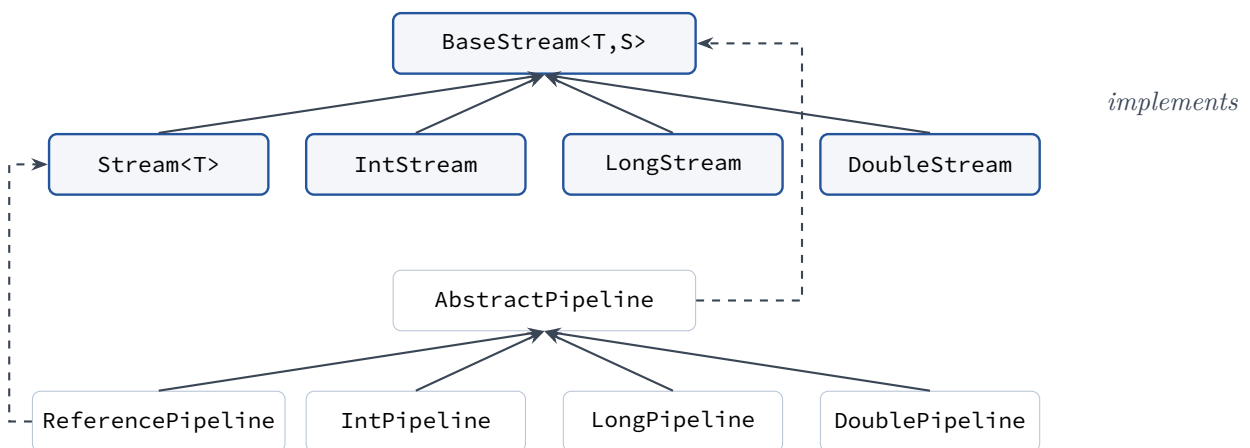
Una parola, e la pipeline gira su più core. Funziona *solo* se le lambda sono pure, cioè senza stato condiviso e senza assumere un ordine, e conviene solo con molti elementi e un lavoro per elemento non banale. Che cosa succede davvero lo vedremo in §3.6.7; usarlo con cognizione richiede le nozioni sui thread, che sono l'argomento di una dispensa a parte. Fino ad allora: sequenziale.

3.6 Dentro la libreria

Tutte le regole della §3.2 (laziness, verticalità, short-circuit, barriere, consumo unico) sono conseguenze di un unico meccanismo, semplice una volta visto. Questa sezione apre `java.util.stream` e lo mostra. Il codice è quello del JDK, semplificato nei nomi generici e nei dettagli che non contano; i nomi delle classi sono quelli veri, così che si possano ritrovare nei sorgenti.

3.6.1 Le interfacce pubbliche

Ciò che il programmatore vede sono interfacce. In cima c'è `BaseStream<T, S>`, che dichiara quello che ogni stream ha in comune: `iterator()` e `spliterator()` per accedere agli elementi, `parallel()` e `sequential()` per scegliere la modalità, `onClose` e `close`, ereditato da `AutoCloseable`. Da lei discendono le quattro interfacce che si usano: `Stream<T>` per gli oggetti e `IntStream`, `LongStream`, `DoubleStream` per i primitivi. Sono quattro e non una perché le firme dei metodi sono diverse: `map` di `Stream<T>` riceve una `Function`, quello di `IntStream` un `IntUnaryOperator`, e `sum` esiste solo sui primitivi.



Head · StatelessOp · StatefulOp: le classi interne di ciascuna pipeline

Le implementazioni stanno nello stesso package e non sono pubbliche: nessun programma le nomina, e possono cambiare da una versione all'altra. `AbstractPipeline` contiene quasi tutta la logica; `ReferencePipeline` e le tre gemelle per i primitivi aggiungono solo le operazioni con la firma giusta per il loro tipo. Quando scriviamo `myList.stream()` chiamiamo un metodo default di `Collection`:

```

default Stream<E> stream() {
    return StreamSupport.stream(spliterator(), false);
}
  
```

Due cose da notare. La collezione non sa nulla degli stream: fornisce solo uno `Spliterator`, e `StreamSupport` costruisce lo stream attorno a esso. E il `false` è “non parallelo”: `parallelStream()` è lo stesso metodo con `true`.

3.6.2 Splitterator: la sorgente

Iterator ha due metodi, `hasNext` e `next`, e chi lo usa tira gli elementi uno alla volta. `Splitterator` **JAVA 8** è il suo omologo pensato per gli stream, con tre differenze: gli elementi vengono *spinti* verso un `Consumer`, si può dividere in due, e dichiara delle proprietà della sorgente.

```
public interface Splitterator<T> {
    boolean tryAdvance(Consumer<? super T> action);    // un elemento, se c'è
    default void forEachRemaining(Consumer<? super T> action) {
        do { } while (tryAdvance(action));
    }
    Splitterator<T> trySplit();                        // metà' degli elementi a un nuovo splitterator
    long estimateSize();
    int characteristics();                             // SIZED, ORDERED, SORTED, DISTINCT, ...
}
```

`tryAdvance` consegna un elemento all'azione e dice se c'era; `forEachRemaining` li consegna tutti. `trySplit` serve al parallelismo: un `ArrayList` si divide a metà con un'aritmetica sugli indici, un `Stream.iterate` non si può dividere affatto, perché ogni elemento dipende dal precedente. Le *caratteristiche* sono un insieme di bit che descrivono la sorgente: `SIZED` se il numero di elementi è noto in anticipo, `ORDERED` se l'ordine conta, `SORTED` e `DISTINCT` se gli elementi sono già ordinati o già distinti. Un `ArrayList` è `SIZED` e `ORDERED`; un `HashSet` è `SIZED` e `DISTINCT`; un `TreeSet` anche `SORTED`; `IntStream.range` è tutte e quattro insieme; `Stream.iterate` solo `ORDERED`. Vedremo tra poco (§3.6.6) che la libreria usa questi bit per saltare lavoro.

3.6.3 La catena degli stadi

Che cosa fa, esattamente, `filter`? Nel JDK, tolti i generici, è questo:

```
public final Stream<T> filter(Predicate<? super T> predicate) {
    return new StatelessOp<T, T>(this, ...) {
        Sink<T> opWrapSink(int flags, Sink<T> downstream) {
            return new Sink.ChainedReference<T, T>(downstream) {
                public void accept(T u) {
                    if (predicate.test(u)) downstream.accept(u);
                }
            };
        }
    };
}
```

`filter` non tocca nessun elemento: crea un oggetto, uno *stadio* (`StatelessOp` è una sottoclasse di `ReferencePipeline`) collegato a `this`, cioè allo stadio precedente, e lo restituisce. Il predicato viene messo da parte, dentro una classe anonima che verrà usata dopo. `map`, `sorted`, `limit` fanno lo stesso: ognuno un nuovo stadio, che ricorda il precedente. Dopo quattro operazioni intermedie abbiamo cinque oggetti in fila, dalla `Head` che tiene lo `Splitterator` fino all'ultimo stadio, ciascuno con un riferimento `previousStage` al proprio predecessore. È una lista concatenata di descrizioni, e questo è tutto ciò che esiste prima dell'operazione terminale: la *laziness* non è una scelta aggiuntiva, è che *non c'è ancora codice che possa lavorare*.

Ogni stadio ha anche un campo booleano `linkedOrConsumed`, che diventa vero quando gli si aggancia uno stadio successivo o quando gli si applica una terminale. Chiamare una seconda operazione su uno stadio già usato trova il campo a `true` e lancia `IllegalStateException` con il messaggio visto a pagina 3: ecco da dove viene “stream has already been operated upon or closed”.

3.6.4 Sink: il momento dell'esecuzione

L'operazione terminale ha di fronte la catena degli stadi e uno `Splitterator` in testa. Deve trasformare le descrizioni in codice che lavora, e lo fa con la seconda interfaccia chiave della libreria:

```
interface Sink<T> extends Consumer<T> {
    default void begin(long size) { }           // sta per arrivare un flusso
    void accept(T t);                           // un elemento (da Consumer)
    default void end() { }                     // il flusso e' finito
    default boolean cancellationRequested() { return false; }
}
```

Un Sink è un Consumer con un inizio, una fine e la possibilità di dire “basta”. Il metodo `opWrapSink` di ogni stadio, che abbiamo visto in `filter`, riceve il sink dello stadio *a valle* e restituisce il proprio, che lo avvolge: il sink di `filter` chiama `downstream.accept` solo se il predicato è vero, quello di `map` chiama `downstream.accept(mapper.apply(u))`. La terminale percorre la catena *all'indietro*, dall'ultimo stadio al primo, avvolgendo ogni volta:

```
// AbstractPipeline, semplificato
Sink<E> wrapSink(Sink<E> sink) {
    for (AbstractPipeline p = this; p.depth > 0; p = p.previousStage)
        sink = p.opWrapSink(p.previousStage.combinedFlags, sink);
    return sink;
}
```

Il risultato è una catena di sink speculari alla catena degli stadi: il sink di `filter` contiene quello di `map`, che contiene quello di `sorted`, che contiene quello della terminale. Poi lo `Splitterator` spinge gli elementi nel primo:

```
// AbstractPipeline, semplificato
void copyInto(Sink<E> sink, Spliterator<E> spliterator) {
    if (!SHORT_CIRCUIT.isKnown(combinedFlags)) {
        sink.begin(spliterator.getExactSizeIfKnown());
        spliterator.forEachRemaining(sink);
        sink.end();
    } else {
        sink.begin(spliterator.getExactSizeIfKnown());
        do { } while (!sink.cancellationRequested() &&
            spliterator.tryAdvance(sink));
        sink.end();
    }
}
```

Qui c'è la spiegazione di tre regole insieme.

- **Verticalità.** `forEachRemaining` consegna un elemento al sink di `filter`; se passa, il sink lo consegna a quello di `map`, che lo trasforma e lo consegna al successivo, fino alla terminale. Solo allora `forEachRemaining` passa all'elemento seguente. Non esiste nessun buffer fra gli stadi: un elemento è un passaggio di chiamate annidate.
- **Short-circuit.** Se nella pipeline c'è un'operazione short-circuit, la libreria non usa `forEachRemaining` ma un ciclo di `tryAdvance`, e prima di ogni elemento chiede alla catena se vuole fermarsi. Il sink di `anyMatch` ha un campo `stop`, lo mette a `true` al primo elemento che soddisfa il predicato e lo restituisce da `cancellationRequested`; quello di `limit(n)` conta e risponde `true` quando ha lasciato passare `n` elementi. Ogni sink intermedio inoltra la domanda a valle: `cancellationRequested() { return downstream.cancellationRequested(); }`. Ecco perché la versione con `forEachRemaining` è riservata alle pipeline senza short-circuit: è più veloce, perché non fa la domanda a ogni elemento.
- **Barriera.** Il sink di `sorted` non inoltra nulla in `accept`: accumula in un `ArrayList`. È in `end()` che ordina la lista e poi, con un ciclo, chiama `downstream.begin`, `downstream.accept` per ogni elemento e `downstream.end`. Tutto ciò che sta a valle di `sorted` vede un flusso che parte quando quello a monte è finito: l'orizzontalità osservata a pagina 6 è questo.

```
// SortedOps, semplificato
final class RefSortingSink<T> extends Sink.ChainedReference<T, T> {
    private ArrayList<T> list;
    public void begin(long size) { list = new ArrayList<>(); }
    public void accept(T t)      { list.add(t); }
    public void end() {
        list.sort(comparator);
        downstream.begin(list.size());
        for (T t : list) downstream.accept(t);
        downstream.end();
    }
}
```

(Nel JDK il ciclo di `end` controlla anche `downstream.cancellationRequested()`, così che un `limit` dopo `sorted` fermi la consegna: l'ordinamento però è già stato fatto per intero.)

3.6.5 Le operazioni terminali

Le terminali sono oggetti `TerminalOp` con due metodi, `evaluateSequential` e `evaluateParallel`. Sono raggruppate in poche classi: `ForEachOps`, `FindOps` (`findFirst`, `findAny`), `MatchOps` (`anyMatch` e sorelle) e `ReduceOps`, che è la più interessante perché serve `reduce`, `collect`, `count`, `min` e `max`. Un `collect(collector)` crea un sink così:

```
// ReduceOps.makeRef(Collector), semplificato
class ReducingSink implements Sink<T> {
    A state;
    public void begin(long size) { state = collector.supplier().get(); }
    public void accept(T t)      { collector.accumulator().accept(state, t); }
    R get()                      { return collector.finisher().apply(state); }
}
```

Le quattro funzioni della Definizione 3.4 sono esattamente i quattro momenti del sink: il `supplier` in `begin`, l'`accumulator` in `accept`, il `finisher` alla fine; il `combiner` entra

in gioco solo in parallelo. `Collectors.toList()` è (`ArrayList::new`, `List::add`, `addAll`, identità); `counting()` è una somma di uni; `groupingBy(f, downstream)` ha per contenitore una `HashMap` e per accumulatore, in sostanza, `map.computeIfAbsent(f.apply(t), k -> downstream.supplier().get())` seguito dall'accumulatore del collettore interno: la stessa cosa che il ciclo del censimento (Esempio 3.2) faceva a mano. `reduce(identità, op)` è lo stesso sink con `state = identità` in `begin` e `state = op.apply(state, t)` in `accept`.

3.6.6 Le bandiere e le scorciatoie

Ogni stadio dichiara che cosa fa alle caratteristiche del flusso: `filter` toglie `SIZED` (non si sa più quanti elementi passeranno), `map` toglie `SORTED` e `DISTINCT` (la trasformazione può rompere l'ordine e creare duplicati) ma conserva `SIZED`, `sorted` aggiunge `SORTED`. Le bandiere si combinano lungo la catena (è il campo `combinedFlags` visto in `wrapSink`), e la terminale le legge per evitare lavoro inutile. Due casi concreti:

- `sorted()` su un flusso già `SORTED` con l'ordine naturale restituisce il sink a valle così com'è: ordinare un `TreeSet` non costa nulla;
- `count()` su un flusso `SIZED` **JAVA 9** restituisce `spliterator.getExactSizeIfKnown()` senza attraversare la pipeline.

Il secondo caso sorprende la prima volta che lo si incontra:

```
long n = List.of("alfa", "beta", "gamma").stream()
    .map(s -> log("Map", s))
    .count();
System.out.println(n);
```

3

Nessuna stampa da `Map`: la lista è `SIZED`, `map` non cambia il numero di elementi, quindi la risposta è nota senza chiamare la lambda. La documentazione di `count` lo dice esplicitamente. Se si aggiunge un `filter`, `SIZED` sparisce e la pipeline viene eseguita. È un altro argomento contro gli effetti collaterali nelle lambda intermedie: la libreria non promette di eseguirle.

3.6.7 Parallelo, in breve

Con `parallel()` la terminale chiama `evaluateParallel`, che divide il lavoro con `trySplit`: lo spliterator viene spaccato a metà, poi ogni metà ancora a metà, finché i pezzi sono abbastanza piccoli (circa quattro per core disponibile). Ogni pezzo diventa un compito eseguito da un thread del `ForkJoinPool` comune della JVM, e su ogni pezzo gira la stessa catena di sink vista sopra, con un proprio contenitore. Alla fine i risultati parziali vengono fusi con il combiner del collettore (o con l'operatore di `reduce`): per `toList` è un `addAll`, per `groupingBy` è la fusione delle mappe gruppo per gruppo. Ecco perché le lambda devono essere pure: girano su thread diversi, in un ordine che dipende dalla divisione, e un contatore condiviso verrebbe incrementato da più thread insieme. Ed ecco perché la sorgente conta: un `ArrayList` o un `range` si dividono in tempo costante e in parti uguali, una `LinkedList` deve scorrere per trovare la metà, un `iterate` non si divide affatto.

3.6.8 Considerazioni sull'efficienza

Con il meccanismo davanti agli occhi si può ragionare su che cosa costa. Il quadro generale è rassicurante: una pipeline sequenziale ben scritta ha la stessa *complessità* del ciclo equivalente, perché ogni elemento passa una volta per ogni stadio. Le differenze sono nelle costanti, e conviene sapere dove stanno.

Il costo fisso. Costruire una pipeline alloca un oggetto per stadio, poi uno spliterator, poi un sink per stadio al momento della terminale. Sono una decina di oggetti piccoli e di breve vita: il garbage collector li smaltisce quasi gratis. Per una lista di cinque elementi un ciclo `for` è più veloce, ma la differenza si misura in centinaia di nanosecondi; per una lista di un milione di elementi il costo fisso sparisce nel rumore. Il caso in cui pesa è una pipeline creata milioni di volte su sorgenti minuscole, per esempio dentro un metodo chiamato per ogni pixel o per ogni pacchetto di rete.

Le chiamate. In un ciclo `for`, il compilatore JIT vede un blocco di codice e lo ottimizza come un tutto. In una pipeline, ogni `accept` è una chiamata attraverso un'interfaccia, e la lambda dentro è un'altra chiamata attraverso un'interfaccia. Il JIT sa eliminare queste chiamate (*inlining*) quando in un punto del programma passa sempre lo stesso tipo di oggetto; ma il ciclo dentro `forEachRemaining` di `ArrayList` è lo stesso per tutte le pipeline del programma, e vede sink di tipi diversi. Il punto di chiamata diventa *megamorfico* e l'*inlining* salta. È la ragione per cui, nei micro-benchmark, lo stream su un array di interi resta più lento del ciclo anche quando i dati sono tanti; ed è una ragione che riguarda i cicli più interni di un programma, non il codice ordinario.

Il boxing. `Stream<Integer>` crea un oggetto `Integer` per ogni elemento a ogni passaggio numerico, e lo disfa subito dopo. Per i calcoli sui numeri gli stream primitivi (§3.3) sono la scelta giusta, e conviene passare a `mapToInt` appena possibile invece di restare in `Stream<Integer>` fino alla fine.

Le barriere. `sorted` alloca una lista di tutti gli elementi e paga un ordinamento $O(n \log n)$ anche se la pipeline poi ne usa uno solo: la libreria *non* trasforma `sorted().limit(k)` in una selezione dei k minimi. Per il minimo o il massimo ci sono `min` e `max`, lineari e senza memoria; per i primi k conviene una struttura dedicata. `distinct` tiene un `HashSet` di tutto ciò che ha già emesso. Se una barriera è inevitabile, va messa il più tardi possibile, dopo i filtri, in modo che riceva pochi elementi.

flatMap e mapMulti. `flatMap` crea uno stream interno per ogni elemento, con il suo spliterator e i suoi stadi, e lo consuma subito. Quando gli elementi sono milioni e ognuno produce pochi valori, il costo fisso dello stream interno domina. `mapMulti` **JAVA 16** risolve il caso: la lambda riceve l'elemento e un `Consumer`, e chiama il consumer per ogni valore da emettere, senza creare stream intermedi:

```
List<String> parole = righe.stream()
    .<String>mapMulti((r, out) -> {
        for (String p : r.split(" ")) out.accept(p);
    })
    .toList();
```

Raccogliere. `toList()` **JAVA 16** chiama `toArray()`, che con una sorgente `SIZED` alloca l'array della dimensione esatta una volta sola, e avvolge l'array in una lista immutabile: nessuna copia. `collect(toList())` fa crescere un `ArrayList` per raddoppi. Anche `groupingBy` con `counting()` è più economico di `groupingBy` seguito dal conteggio delle liste: il primo non crea le liste.

Il parallelismo. La divisione e la fusione hanno un costo, e il pool è condiviso con il resto del programma. Il parallelismo paga quando il numero di elementi è grande (decine di migliaia, almeno), il lavoro per elemento è sensibile (non una somma), la sorgente si divide bene e il collettore fonde a buon mercato (una lista, non un `joining` di stringhe). Fuori da queste condizioni è più lento del sequenziale.

Osservazione 3.3

Nessuno di questi punti cambia l'ordine di grandezza del tempo di esecuzione, e nessuno vale la pena di essere considerato prima di aver misurato. Le scelte che contano sono le stesse di un ciclo: filtrare presto, non ordinare quel che non serve, non riallocare a ogni giro. Gli stream le rendono visibili nell'ordine delle righe, e questo è il loro vantaggio anche dal punto di vista dell'efficienza.

3.7 Riepilogo

- Una **pipeline** è sorgente, operazioni intermedie e una terminale. Uno stream non è una collezione: non contiene dati, li fa passare, una volta sola.
- Le intermedie sono **lazy**: descrivono; è la terminale a eseguire. L'elaborazione è **verticale**, un elemento alla volta lungo tutta la pipeline; l'**ordine delle operazioni** determina il lavoro; le operazioni **short-circuit** fermano il flusso appena possibile; **sorted** è una **barriera**.
- Sorgenti: collezioni, valori, array, `range`, `iterate`, `file`. Uno stream infinito va chiuso da `limit`, `takeWhile` o una short-circuit. `IntStream` e sorelle evitano il boxing; `Optional` dice nel tipo che il risultato può mancare.
- Il risultato si costruisce con `collect` e i `Collectors` (`toList`, `toSet`, `joining`, `groupingBy`, `partitioningBy`, `toMap`), con `reduce` o con le riduzioni pronte; `flatMap` appiattisce.
- Dentro: ogni intermedia crea uno **stadio** collegato al precedente; la terminale avvolge la catena in **sink** annidati e lo **spliterator** spinge gli elementi nel primo. Da questo seguono laziness, verticalità, short-circuit (`cancellationRequested`) e barriere (`end`). Le **bandiere** permettono scorciatoie come `count` senza attraversamento.
- Efficienza: stessa complessità del ciclo; costanti più alte per il costo fisso, le chiamate e il boxing; barriere e `flatMap` sono i punti da guardare; `parallel` solo con molti elementi e lavoro per elemento.

3.8 Esercizi

Esercizio 3.1: Dal ciclo alla pipeline

Riscrivere come pipeline il metodo seguente, che restituisce le lunghezze distinte, in ordine crescente, delle parole più lunghe di tre lettere.

```
static List<Integer> lunghezze(List<String> parole) {  
    Set<Integer> viste = new TreeSet<>();  
    for (String p : parole)  
        if (p.length() > 3) viste.add(p.length());  
    return new ArrayList<>(viste);  
}
```

Esercizio 3.2: Tracciare l'esecuzione

Senza eseguire, scrivere l'output esatto della pipeline seguente (`log` è il metodo di pagina 3) e spiegare quali regole lo determinano.

```
Stream.of("uno", "due", "tre", "quattro", "cinque")  
    .filter(s -> log("F", s).length() == 3)  
    .map(s -> log("M", s).toUpperCase())  
    .limit(2)  
    .forEach(s -> log("E", s));
```

Esercizio 3.3: Numeri

Con uno stream primitivo calcolare la somma dei quadrati dei numeri dispari da 1 a 99, e poi la media dei numeri da 1 a 100 divisibili per 7. Indicare il tipo del risultato di ciascuna espressione.

Esercizio 3.4: Raggruppare

Data una `List<String>` di parole, costruire con una sola pipeline ciascuna delle mappe seguenti: (a) da lunghezza a lista delle parole con quella lunghezza; (b) da lunghezza a numero di parole; (c) da lunghezza alla parola più lunga in ordine alfabetico fra quelle di quella lunghezza; (d) la partizione fra parole che iniziano per vocale e le altre.

Esercizio 3.5: Appiattare

Data una `List<String>` di frasi, ottenere l'elenco delle parole distinte in ordine alfabetico, ignorando maiuscole e minuscole. Riscrivere poi la soluzione con `mapMulti` al posto di `flatMap`.

Esercizio 3.6: Che cosa stampa, e perché

Prevedere l'output dei due frammenti e spiegare la differenza con ciò che si è visto in §3.6.

```
long a = Stream.of("x", "y", "z").peek(s -> System.out.println("A " +
    s)).count();
long b = Stream.of("x", "y", "z").peek(s -> System.out.println("B " + s))
    .filter(s -> true).count();
```

Esercizio 3.7: Un collettore a mano

Con `Collector.of(supplier, accumulator, combiner, finisher)` scrivere un collettore `Collector<String, ?, Integer>` che calcola la somma delle lunghezze delle stringhe raccolte, e usarlo su `myList`. Suggerimento: come contenitore va bene un `int[]` di un elemento.

Soluzioni

Soluzione 3.1.

```
static List<Integer> lunghezze(List<String> parole) {
    return parole.stream()
        .filter(p -> p.length() > 3)
        .map(String::length)
        .distinct()
        .sorted()
        .toList();
}
```

Il `TreeSet` faceva due cose insieme, eliminare i duplicati e ordinare; nella pipeline sono due operazioni distinte, `distinct` e `sorted`, entrambe dopo il filtro, così che lavorino sui soli elementi che contano. Con `mapToInt` al posto di `map` si eviterebbe il boxing, ma il risultato dev'essere una `List<Integer>`, quindi alla fine servirebbe comunque `boxed()`.

Soluzione 3.2.

```
F: uno
M: uno
E: UNO
F: due
M: due
E: DUE
```

Ogni elemento attraversa tutta la pipeline prima del successivo (verticalità), per cui le stampe sono raggruppate per elemento e non per fase. `limit(2)` è short-circuit: dopo aver lasciato passare due elementi risponde `true` a `cancellationRequested`, e la sorgente smette di consegnare: tre, quattro e cinque non arrivano nemmeno al filtro.

Soluzione 3.3.

```
int somma = IntStream.rangeClosed(1, 99)
    .filter(n -> n % 2 == 1)
    .map(n -> n * n)
    .sum(); // int: 166650

OptionalDouble media = IntStream.rangeClosed(1, 100)
    .filter(n -> n % 7 == 0)
    .average(); // OptionalDouble[52.5]
```

sum su un IntStream restituisce int (zero se lo stream è vuoto: la somma ha un'identità); average restituisce OptionalDouble, perché la media di zero numeri non esiste. Per i quadrati di numeri grandi conviene mapToLong prima del sum, per non superare il massimo di int.

Soluzione 3.4.

```
Map<Integer, List<String>> a = parole.stream()
    .collect(groupingBy(String::length));
Map<Integer, Long> b = parole.stream()
    .collect(groupingBy(String::length, counting()));
Map<Integer, Optional<String>> c = parole.stream()
    .collect(groupingBy(String::length, maxBy(Comparator.naturalOrder())));
Map<Boolean, List<String>> d = parole.stream()
    .collect(partitioningBy(p -> "aeiou".indexOf(p.charAt(0)) >= 0));
```

In (c) il collettore interno maxBy restituisce un Optional, perché in generale un gruppo potrebbe essere vuoto; qui non può esserlo (un gruppo esiste solo se ha almeno un elemento), e con collectingAndThen(maxBy(...), Optional::get) si può togliere l'Optional dal tipo della mappa.

Soluzione 3.5.

```
List<String> parole = frasi.stream()
    .flatMap(f -> Arrays.stream(f.split("\\s+")))
    .map(String::toLowerCase)
    .distinct()
    .sorted()
    .toList();
```

Con mapMulti la prima operazione diventa

```
.<String>mapMulti((f, out) -> {
    for (String p : f.split("\\s+")) out.accept(p);
})
```

Il tipo esplicito <String> serve perché il compilatore non può dedurre dal solo Consumer che cosa verrà emesso. toLowerCase va prima di distinct, altrimenti “Alfa” e “alfa” sarebbero due parole diverse.

Soluzione 3.6. Il primo frammento stampa nulla e `a` vale 3: `Stream.of` ha una sorgente `SIZED`, `peek` conserva la bandiera, e `count` risponde con la dimensione dello spliterator senza attraversare la pipeline. Il secondo stampa `B x`, `B y`, `B z` e `b` vale 3: `filter` toglie `SIZED`, quindi `count` deve contare gli elementi che passano, e per farlo li fa scorrere tutti, `peek` compreso. La morale è che `peek` serve a osservare una pipeline durante la messa a punto, non a produrre effetti su cui il programma conta.

Soluzione 3.7.

```
Collector<String, int[], Integer> sommaLunghezze = Collector.of(
    () -> new int[1],                // supplier
    (acc, s) -> acc[0] += s.length(), // accumulator
    (x, y) -> { x[0] += y[0]; return x; }, // combiner
    acc -> acc[0]);                  // finisher

int totale = myList.stream().collect(sommaLunghezze); // 10
```

Il contenitore è un array perché l'accumulatore deve *modificarlo*, e un `Integer` è immutabile. Il combiner serve solo in parallelo, ma è obbligatorio: somma i due parziali. Lo stesso risultato si ottiene in una riga con `myList.stream().mapToInt(String::length).sum()`, che è la forma da preferire: scrivere un collettore a mano ha senso quando nessuna combinazione di `Collectors` esprime l'accumulo che serve.